

Problema de la Mochila (Knapsack Problem)

1. Definición del problema

El problema de la mochila, conocido en inglés como Knapsack problem, es uno de los 21 problemas clásicos np-completos propuestos por Richard Karp. En síntesis se trata de, a partir de un conjunto de objetos conocido como “n”, donde cada uno tiene un peso “w” asignado y un valor “v” asignado, llenar una mochila sin que supere el total de su capacidad, definida como “c”. El objetivo de este problema es encontrar la combinación que permita maximizar la utilidad de los objetos, sin incumplir las restricciones de peso.

Es importante destacar que el problema de la mochila trasciende distintas áreas a través de su aplicación. Destacando la logística, optimización y asignación de recursos, donde las decisiones tomadas también van sujetas a restricciones que cumplir.

2. Entrada y salida

2.1. Entrada

- n_i : conjunto de objetos
- v_i : valor de cada objeto
- w_i : peso de cada objeto
- C : capacidad total de la mochila

2.2. Salida

Vector binario $x_i \in \{0, 1\}$, donde cada componente indica si el objeto fue seleccionado o no. En notación matemática pura se escribe de la siguiente forma:

$$x \in \{0, 1\}^n$$

3. Ejemplo clásico

Parte de la naturaleza de este problema y de muchos que forman parte de los 21 problemas np-completos es emular o simular situaciones reales. Por ejemplo, imaginemos que tenemos un viaje súper importante a una convención urbana donde queremos vender algunos de nuestros objetos, sin embargo, como equipaje, sólo podemos llevar una mochila que posee una capacidad máxima de 6 kg.

Dentro de los objetos que deseamos llevar se encuentran una laptop, un PlayStation 4, unas Jordan Retro 4, un balón de básquet y una gorra New Era, que pesan 2 kg, 0.5 kg, 1 kg, 3 kg y 0.75 kg respectivamente, que se venderán a un valor de 10, 6, 5, 3 y 2 respectivamente.

Bajo este contexto, para obtener la mayor cantidad de ganancias, debemos maximizar la utilidad de los objetos que podemos llevar sin exceder la capacidad total de la mochila.

Este ejemplo básico e ilustrativo, sintetiza la esencia del problema de la mochila. Tomar decisiones en base a maximizar la utilidad de los objetos, respetando las restricciones de capacidad impuestas.

- Laptop: 2 kg, \$10
- PlayStation 4: 0.5 kg, \$6
- Jordan Retro 4: 1 kg, \$5
- Balón de básquet: 3 kg, \$3
- Gorra New Era: 0.75 kg, \$2

El objetivo es maximizar la ganancia sin exceder los 6 kg.

4. Modelo matemático

4.1. Parámetros

- n : número de objetos.
- v_i : valor del objeto i con $i = 1, \dots, n$
- w_i : peso del objeto i con $i = 1, \dots, n$
- C : capacidad de la mochila.

4.2. Variables de decisión

$x \in \{0, 1\}$ indica si el objeto es seleccionado (1) o no es seleccionado (0).

4.3. Restricción

Subject to $\rightarrow \sum_{i=1}^n w_i x_i \leq C$

4.4. Función objetivo

Maximize $\rightarrow \sum_{i=1}^n v_i x_i$

4.5. Versión de decisión (binaria)

$x_i \in \{0, 1\}, i = 1, \dots, n$

5. Complejidad

Ya sabemos que la complejidad de este problema es NP- Completo, sin embargo, el por qué es así es porque cumple con las dos condiciones: estar clasificado como NP y ser tan difícil como los demás (NP-Hard).

Se dice que el problema pertenece a NP porque, dada una solución candidata, es posible verificar en tiempo polinomial si cumple con las restricciones del problema y calcular su valor total, es decir, alguien nos puede dar los objetos que eligió y a partir de esto podemos calcular los pesos y valor de los objetos en tiempo polinomial $[O(n), O(n^2), O(n^3), O(n^K)]$. Por otra parte, cumple con su naturaleza de NP-Hard, que significa que otros problemas difíciles pueden ser reducidos a Knapsack en tiempo polinomial, como por ejemplo, subset sum.

6. Significado NP

NP o Non-deterministic Polynomial Time en inglés es una clase de complejidad que sirve para clasificar problemas cuyas soluciones pueden verificarse en tiempo polinomial por una máquina determinista, es decir, son aquellos problemas en los cuales es fácil verificar si una solución es correcta, sin embargo, no quiere decir que sea fácil llegar a esa solución

7. Relación con otros problemas

Knapsack, al igual que los 21 problemas listados como NP-Completo guardan relación entre sí, ya que todos los problemas clasificados como NP pueden ser reducidos a NP-Completo en tiempos polinomiales, estableciendo equivalencia en temas de complejidad.

Profundizando en Subset Sum, cuyo objetivo es, dado un conjunto finito de números positivos y un valor objetivo, determinar si existe un subconjunto que, sumados sus valores, den exactamente igual al valor objetivo.

Este problema, aunque no hable explícitamente de pesos ni valores, si guarda una similitud especial, pues al hablar de subconjuntos y suma objetivo, estamos hablando implícitamente de pesos y capacidad. Por lo que, de manera natural, Subset Sum podría transformarse en Knapsack tranquilamente.

8. Implementación

Como algoritmos, se utilizó backtracking como solución base y para obtener soluciones óptimas utilizamos programación dinámica con memorización.

La programación dinámica consiste en dividir problemas en subproblemas más chicos, cuyas soluciones permitan resolver el problema grande. En este caso, para el problema de la mochila, buscamos maximizar el valor total de los objetos que llevamos, siendo este nuestro problema global. Para descomponerlo, se generan subproblemas definidos por el número de objetos y la capacidad disponible de la mochila, dentro de estos se evalúan la decisión de si incluir los objetos o no.

De esta forma obtenemos una solución pequeña, que al sumar sus valores individuales maximiza la utilidad, solucionando el problema global.

La memorización o memoización es una técnica que nos permite almacenar soluciones ya existentes, para así evitar que el algoritmo las calcule de nuevo. Para esto almacenamos en un diccionario llamado “memoria” la capacidad total del objeto y el número de objetos, de esta forma, si recibimos un input que sea igual a alguna solución ya almacenada, el algoritmo ocupará esa solución, reduciendo considerablemente el tiempo de ejecución.

El backtracking es un tipo de algoritmo que permite explorar espacios de solución pero haciendo podas mediante llamadas recursivas, en este caso, establecemos parámetros como peso acumulado o valor acumulado para guiar la exploración, tomando lugar la poda cuando la solución parcial excede la capacidad de la mochila, evitando explorar caminos que no cumplen con la restricción del problema.

9. Análisis de resultados

En base a lo observado, es posible realizar una comparación en términos de eficiencia de los tiempos de ejecución entre ambos algoritmos, obteniendo distintos resultados según el lenguaje en que se implementen.

En general, al comparar C++, Java y Python, se observa que C++ presenta los menores tiempos de ejecución, seguido de Java y, finalmente, Python. Esto se debe a que C++ es un lenguaje compilado que genera código cercano al hardware, permitiendo una ejecución más eficiente.

Por su parte, Java se ejecuta sobre la máquina virtual JVM, la cual introduce una capa adicional de abstracción. Sin embargo, mediante el uso del compilador JIT, el código es optimizado y transformado a código máquina durante la ejecución, mejorando su rendimiento.

Finalmente, Python, al ser un lenguaje interpretado, presenta un mayor overhead asociado a la ejecución de instrucciones, llamadas a funciones y acceso a estructuras de datos como diccionarios, lo que impacta negativamente en los tiempos de ejecución en comparación con los otros lenguajes.

9.1. Python

En base a los resultados obtenidos en Python, los tiempos de ejecución de la programación dinámica se mantienen consistentemente bajos en cada ejemplo, a diferencia del algoritmo de backtracking, cuyos tiempos presentan variaciones más pronunciadas, alcanzando picos elevados en algunos casos, como ocurre en el ejemplo 5.

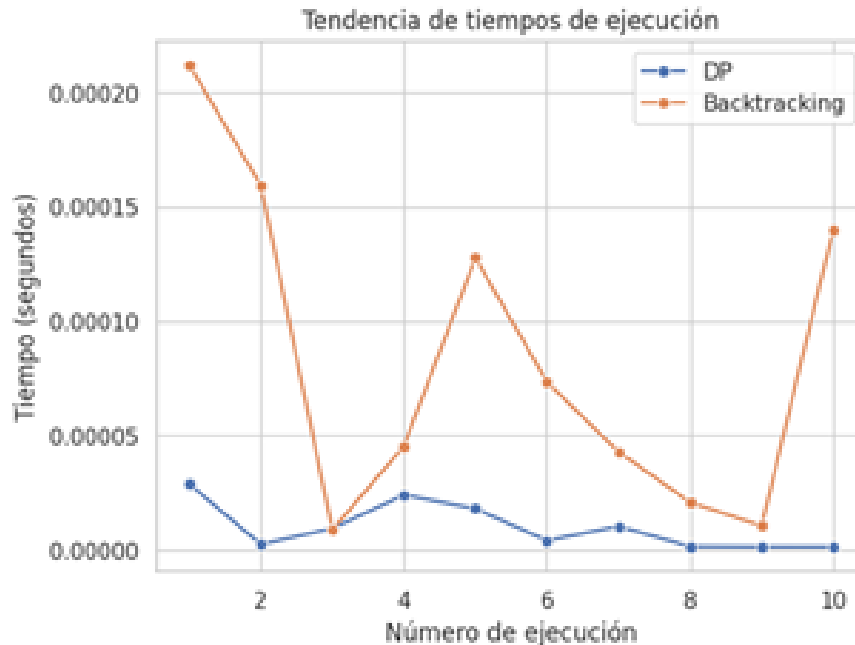


Figura 1: Tendencia de tiempos de ejecución python.

Esta diferencia se explica por el enfoque de ambos algoritmos. Mientras la programación dinámica descompone el problema global en subproblemas y utiliza memorización para almacenar resultados previamente calculados, evitando recomputaciones, el backtracking explora todas las combinaciones posibles. Esto se traduce en una complejidad temporal de $O(2^N)$ para backtracking, en contraste con la complejidad $O(nC)$ de programación dinámica.

Sin embargo, es importante destacar que, antes de cada ejecución, la memoria utilizada por el algoritmo de programación dinámica es reiniciada mediante una variable global. Esto se debe a que, en cada ejecución, las instancias de p , v y C varían al generarse de forma

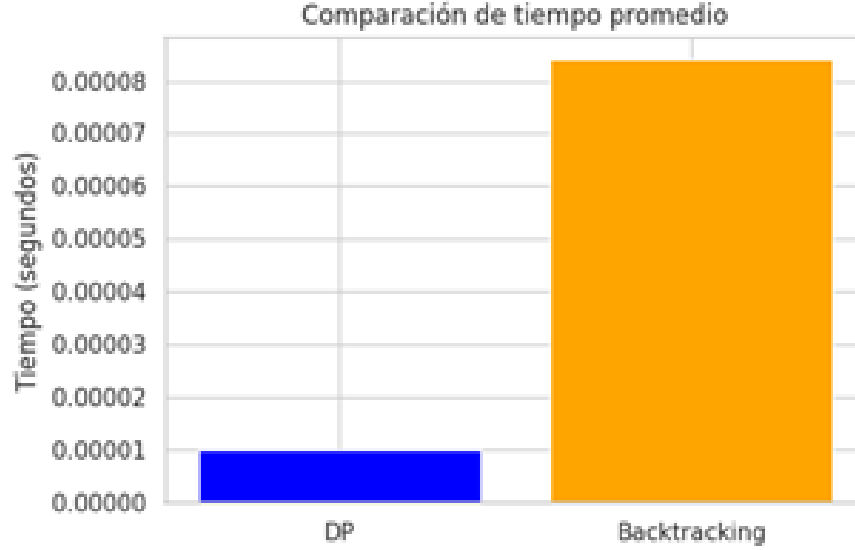


Figura 2: Tiempos promedio Python.

aleatoria, lo que podría provocar resultados incorrectos si se reutilizan valores previamente almacenados.

Aunque esta reinicialización puede influir levemente en los tiempos de ejecución observados, no afecta la complejidad temporal teórica del algoritmo, la cual se mantiene en $O(nC)$ por instancia.

---	Ejemplo	n	Capacidad	Tiempo_DP	Resultado_DP	Tiempo_BT	Resultado_BT
0	1	9	25	0.000029	72	0.000212	85
1	2	9	26	0.000002	89	0.000159	94
2	3	7	13	0.000009	36	0.000009	32
3	4	10	12	0.000034	37	0.000045	50
4	5	9	30	0.000018	74	0.000128	68
5	6	8	27	0.000004	61	0.000073	65
6	7	7	28	0.000010	67	0.000043	80
7	8	6	26	0.000001	53	0.000021	65
8	9	5	11	0.000001	28	0.000010	35
9	10	9	26	0.000001	89	0.000140	89

Figura 3: Resultados Python.

9.2. C++

En el caso de C++ los resultados fueron diferentes, esto es debido a el overhead causado por la implementación de estructuras como map y pair que se genera una complejidad temporal de $O(\log M)$, donde M corresponde a la estructura de datos almacenados. Además las múltiples consultas y creación de claves compuestas mediante pair también aumentan el tiempo de ejecución, volviendo a la programación dinámica una implementación temporalmente más pesada.



Figura 4: Tendencia de tiempos de ejecución C++.

Aunque si comparamos complejidades teóricas, siempre la programación dinámica será más rápida que backtracking, en lenguajes como C++ el tamaño de la entrada juega un rol más importante, especialmente cuando el n° de objetos es reducido ($N \leq 10$), a partir de esto, el overhead asociado a la memorización no justifica la implementación de recomputaciones, volviendo a backtracking como un algoritmo temporalmente efectivo.

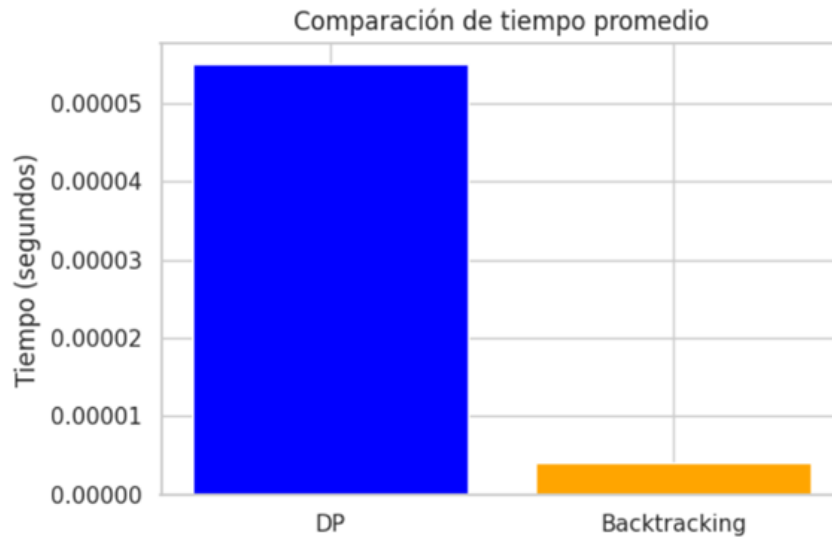


Figura 5: Tiempo promedio de ejecución C++.

```

=== DP ===
Ejemplo 1 | n=9 | C=25 | Tiempo: 0.000096 s | Resultado: 85
Ejemplo 2 | n=9 | C=26 | Tiempo: 0.000088 s | Resultado: 94
Ejemplo 3 | n=7 | C=13 | Tiempo: 0.000007 s | Resultado: 32
Ejemplo 4 | n=10 | C=12 | Tiempo: 0.000035 s | Resultado: 50
Ejemplo 5 | n=9 | C=30 | Tiempo: 0.000094 s | Resultado: 68
Ejemplo 6 | n=8 | C=27 | Tiempo: 0.000055 s | Resultado: 65
Ejemplo 7 | n=7 | C=28 | Tiempo: 0.000054 s | Resultado: 80
Ejemplo 8 | n=6 | C=26 | Tiempo: 0.000025 s | Resultado: 65
Ejemplo 9 | n=5 | C=11 | Tiempo: 0.000010 s | Resultado: 35
Ejemplo 10 | n=9 | C=26 | Tiempo: 0.000085 s | Resultado: 89

=== Backtracking ===
Ejemplo 1 | n=9 | C=25 | Tiempo: 0.000007 s | Resultado: 85
Ejemplo 2 | n=9 | C=26 | Tiempo: 0.000008 s | Resultado: 94
Ejemplo 3 | n=7 | C=13 | Tiempo: 0.000001 s | Resultado: 32
Ejemplo 4 | n=10 | C=12 | Tiempo: 0.000003 s | Resultado: 50
Ejemplo 5 | n=9 | C=30 | Tiempo: 0.000007 s | Resultado: 68
Ejemplo 6 | n=8 | C=27 | Tiempo: 0.000004 s | Resultado: 65
Ejemplo 7 | n=7 | C=28 | Tiempo: 0.000002 s | Resultado: 80
Ejemplo 8 | n=6 | C=26 | Tiempo: 0.000001 s | Resultado: 65
Ejemplo 9 | n=5 | C=11 | Tiempo: 0.000001 s | Resultado: 35
Ejemplo 10 | n=9 | C=26 | Tiempo: 0.000007 s | Resultado: 89

```

Figura 6: Resultados C++.

9.3. Java

Para la implementación en Java ocurre un comportamiento similar, donde el uso de estructuras como HashMap introduce overhead adicional. En particular, la construcción de claves mediante String implica operaciones de concatenación, creación de objetos y cálculo de hash en cada llamada recursiva, lo que incrementa el costo temporal del algoritmo.

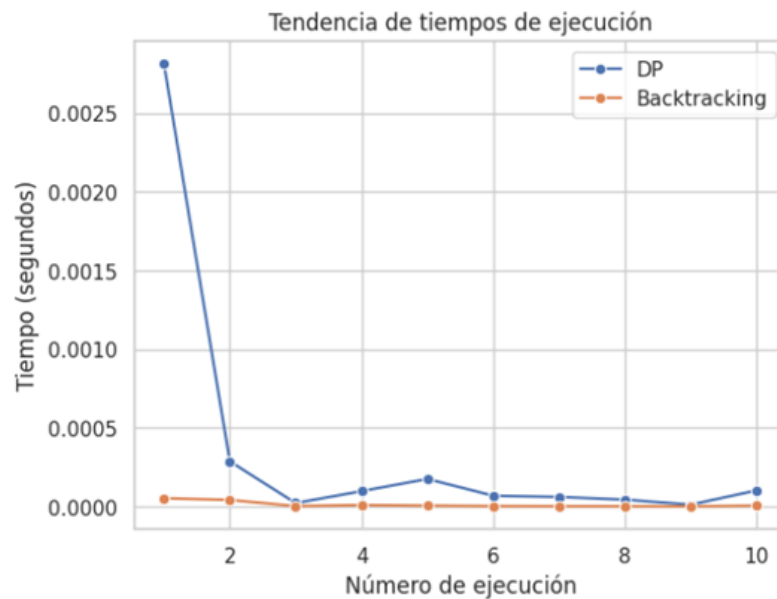


Figura 7: Tendencia en tiempos de ejecución JAVA.

Por otra parte, la naturaleza orientada a objetos de Java implica un mayor uso de memoria debido a la creación de instancias, lo que genera un costo adicional. Además, al ejecutarse

sobre la JVM, se introduce una capa de abstracción que, si bien permite optimizaciones mediante el compilador JIT, también contribuye al overhead general del sistema.



Figura 8: Tiempo promedio por ejecución JAVA.

En consecuencia, para instancias pequeñas $n \leq 10$, este overhead puede superar los beneficios de la memorización, haciendo que el algoritmo de backtracking, al ser más liviano y requerir menos estructuras adicionales, presente mejores tiempos de ejecución en la práctica.

=== DP ===					
Ejemplo 1	n=9	C=25	Tiempo: 0.002815 s	Resultado: 85	
Ejemplo 2	n=9	C=26	Tiempo: 0.000288 s	Resultado: 94	
Ejemplo 3	n=7	C=13	Tiempo: 0.000022 s	Resultado: 32	
Ejemplo 4	n=10	C=12	Tiempo: 0.000098 s	Resultado: 50	
Ejemplo 5	n=9	C=30	Tiempo: 0.000176 s	Resultado: 68	
Ejemplo 6	n=8	C=27	Tiempo: 0.000069 s	Resultado: 65	
Ejemplo 7	n=7	C=28	Tiempo: 0.000061 s	Resultado: 80	
Ejemplo 8	n=6	C=26	Tiempo: 0.000044 s	Resultado: 65	
Ejemplo 9	n=5	C=11	Tiempo: 0.000012 s	Resultado: 35	
Ejemplo 10	n=9	C=26	Tiempo: 0.000103 s	Resultado: 89	
=== Backtracking ===					
Ejemplo 1	n=9	C=25	Tiempo: 0.000053 s	Resultado: 85	
Ejemplo 2	n=9	C=26	Tiempo: 0.000042 s	Resultado: 94	
Ejemplo 3	n=7	C=13	Tiempo: 0.000003 s	Resultado: 32	
Ejemplo 4	n=10	C=12	Tiempo: 0.000010 s	Resultado: 50	
Ejemplo 5	n=9	C=30	Tiempo: 0.000006 s	Resultado: 68	
Ejemplo 6	n=8	C=27	Tiempo: 0.000003 s	Resultado: 65	
Ejemplo 7	n=7	C=28	Tiempo: 0.000002 s	Resultado: 80	
Ejemplo 8	n=6	C=26	Tiempo: 0.000002 s	Resultado: 65	
Ejemplo 9	n=5	C=11	Tiempo: 0.000001 s	Resultado: 35	
Ejemplo 10	n=9	C=26	Tiempo: 0.000006 s	Resultado: 89	

Figura 9: Resultados JAVA.

...	Ejemplo	n	Pesos	Valores	Capacidad
	1	9	[6, 4, 8, 4, 3, 5, 5, 1, 1]	[6, 12, 18, 19, 4, 3, 17, 3, 15]	25
	2	9	[8, 2, 3, 5, 7, 4, 4, 1, 3]	[4, 17, 14, 12, 12, 9, 10, 20, 7]	26
	3	7	[6, 8, 9, 9, 8, 6, 4]	[18, 6, 17, 12, 13, 10, 14]	13
	4	10	[7, 1, 8, 5, 8, 5, 3, 3, 1, 9]	[17, 18, 14, 12, 2, 11, 1, 14, 1, 19]	12
	5	9	[9, 2, 4, 4, 10, 7, 10, 3, 5]	[4, 8, 5, 19, 12, 20, 1, 4, 9]	30
	6	8	[10, 10, 1, 4, 6, 2, 2, 7]	[16, 17, 7, 3, 12, 17, 8, 2]	27
	7	7	[1, 3, 6, 4, 7, 9, 6]	[16, 2, 19, 6, 17, 8, 20]	28
	8	6	[7, 8, 2, 5, 6, 10]	[18, 12, 13, 19, 15, 11]	26
	9	5	[4, 4, 4, 3, 1]	[17, 4, 9, 1, 9]	11
	10	9	[9, 1, 7, 3, 7, 5, 1, 5, 4]	[13, 5, 17, 6, 12, 12, 20, 9, 20]	26

Figura 10: Casos de uso para todos los lenguajes.

10. Escalabilidad

En este punto es donde se observa la principal diferencia en términos de escalabilidad, siendo la programación dinámica la que destaca frente a la implementación de backtracking. Debido a la complejidad temporal del backtracking y su naturaleza, que implica explorar todos los caminos o posibles combinaciones, su tiempo de ejecución crece exponencialmente a medida que aumenta el tamaño de la entrada.

En particular, para valores de $n \leq 15$, la división del problema en subproblemas junto con el almacenamiento de resultados en memoria permite a la programación dinámica reducir significativamente el tiempo total de ejecución.

Por lo tanto, se concluye que, para entradas pequeñas, la implementación de backtracking puede ser aceptable, mientras que, para entradas grandes, es recomendable utilizar programación dinámica, ya que su enfoque basado en subproblemas permite alcanzar una mejor escalabilidad y una complejidad temporal considerablemente menor.

11. Conclusiones

Finalmente, tras implementar y ejecutar ambos algoritmos en tres lenguajes distintos, se observa que el rendimiento no depende únicamente del lenguaje utilizado, sino también de cómo se implementa el algoritmo. Aunque C++ y Java son lenguajes compilados y, en general, más eficientes que Python, el uso de estructuras como map, pair o HashMap introduce overhead adicional asociado a la construcción de claves, acceso a memoria y manejo de objetos.

En contraste, si bien Python presenta mayor overhead debido a su naturaleza interpretada, su implementación puede resultar relativamente más liviana en ciertos casos, generando la aparente sensación de que el algoritmo de backtracking es más eficiente en términos de tiempo.

Sin embargo, esta percepción se debe principalmente al tamaño reducido de las instancias analizadas. Cuando el número de objetos es pequeño, el costo asociado a la memorización

en programación dinámica puede superar sus beneficios, especialmente en lenguajes donde el acceso a estructuras de datos es más costoso.

Por lo tanto, se concluye que, si bien el backtracking puede ser competitivo en instancias pequeñas, su complejidad exponencial lo vuelve inviable para problemas de mayor tamaño. En estos casos, la programación dinámica sigue siendo la estrategia más eficiente, ya que reduce significativamente el número de subproblemas mediante el uso de memorización.

12. Bibliografía

- <https://www.uaeh.edu.mx/scige/boletin/tlahuelilpan/n6/e2.html>
- https://cp-algorithms.com/dynamic_programming/knapsack.html
- <https://arpitbhayani.me/blogs/genetic-knapsack/>
- <https://www.cs.umd.edu/class/spring2025/cmsc451-0101/Lects/lect21-apx-sub-sum.pdf>
- https://www.w3schools.com/dsa/dsa_ref_dynamic_programming.php
- https://www.onlinegdb.com/online_c_plus_compiler
- https://es.wikipedia.org/wiki/Problema_de_las_mochilas